1.1

- An abstract class is a class that acts like the blueprint for other classes; and unlike inheritance, it ensures consistent structure for all subclasses. In order to make a method “abstract”, you must import and use “@abstractmethod” right above the method, and place that method into a class that inherits from “ABC” (abstract base class).
- Partial Abstraction provides ready-to-use methods, alongside others that need to be override the abstract per subclass. But Full Abstraction provides all its methods requiring overriding per subclass.
- No, Python doesn’t have a dedicated interface keyword; but developers achieve the same goal via Abstraction or duck typing.
- It raises an Error; to solve it, you must override the abstract.

1.2

- Duck Typing is a type system used in dynamic programming languages like python. Where it checks for the presence of a given method or attribute instead of the class or type. The name Duck Typing comes from the phrase: *"If it looks like a duck and quacks like a duck, it's a duck"*.
- It allows different objects to be used interchangeably by focusing on what they do instead of what they are.
- Method Overriding requires an official inheritance to a specific parent class, but Duck typing doesn’t require that, it only looks for the method it needs in different objects.
- An example is when wanting to make the code faster and more flexible. If we have different classes that have some common methods (many vehicles that have the “move” or “flash” methods), we can just use any vehicle class that has the “flash” method that we need, instead of looking for the specific vehicle, we are dealing with, to use its “flash” method.